

MINOR PROJECT REPORT

On

SOFTWARE SOLUTIONS TO IDENTIFY USERS BEHIND TELEGRAM, WHATSAPP AND INSTAGRAM BASED DRUG TRAFFICKING

In partial fulfillment of requirements for the degree of
**BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE &
ENGINEERING**

Group ID: [D-G4]

Branch & Section: BTECH & [D]

Submitted by:

Nancy Chouhan – UI/UX & Frontend Support

Mayank Yadav – Backend & ML Engineer

Nandini Soni – Frontend Developer

Manas Patidar – Database Engineer

Under the guidance of
Ms. Shubhangi Shahi



Jan – June 2026

DECLARATION

We hereby declare that the work presented in the project entitled "Software Solutions to Identify Users Behind Telegram, WhatsApp and Instagram Based Drug Trafficking," submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Science & Engineering, is an authentic record of our own work carried out under the supervision and guidance of our Project Guide. The matter embodied in this project has not been submitted for the award of any other degree or diploma.

Date: _____

Student Signature:

Nancy Chouhan -

Mayank Yadav -

Nandini Soni -

Manas Patidar -

PROJECT APPROVAL SHEET

The following team has done the appropriate work related to the “Software Solutions to Identify Users Behind Telegram, WhatsApp and Instagram Based Drug Trafficking” in partial fulfillment for the award of Bachelor of Technology in Computer Science & Engineering, of “SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY” and is being submitted to SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE.

Team:

1. NANCY CHOUHAN [23100BTCSE14974]
2. MAYANK YADAV [23100BTCSE14960]
3. NANDINI SONI [23100BTCSE14975]
4. MANAS PATIDAR [23100BTCSE14953]

Internal Examiner

External Examiner

Date:

CERTIFICATE

This is to certify that the team of Group ID [D-G4], comprising Nancy Chouhan , Mayank Yadav, Nandini Soni, and Manas Patidar, has satisfactorily completed the Minor Project entitled "Software Solutions to Identify Users Behind Telegram, WhatsApp and Instagram Based Drug Trafficking" under the guidance of the Project Guide in partial fulfillment of the degree of Bachelor of Technology in Computer Science & Engineering awarded by SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY affiliated to SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE during the academic year Jan–June 2026.

Ms. Shubhangi Shahi

Project Guide

Mr. Neeraj Arya

Project Coordinator

Dr. Jigyasu Dubey

Head, Dept. of CSE, SVIIT-SVVV
science & engineering
SVIIT-SVVV

Dr. Anand Rajawat

Director, SVIIT-

ACKNOWLEDGEMENTS

We are deeply grateful to all those who supported us throughout this project. We sincerely thank our Project Guide for providing constant guidance, technical direction, and constructive feedback that shaped the direction of this work. His expertise and encouragement were invaluable at every stage.

We also express our sincere gratitude to the Head of the Department of Computer Science & Engineering, the Project Coordinator, and the Director of SVIIT-SVVV for their continued support and for providing us with the necessary infrastructure and resources.

We extend our thanks to all faculty members and non-teaching staff of the Department who were always helpful and cooperative. Our gratitude also goes to our parents, family members, classmates, and friends for their unwavering motivation and valuable suggestions throughout this journey.

It has been a privilege to study at SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE.

ABSTRACT

The rapid proliferation of end-to-end encrypted messaging platforms such as Telegram, WhatsApp, and Instagram has created significant challenges for law enforcement agencies attempting to combat drug trafficking. These platforms offer features like anonymous group chats, disappearing messages, and pseudonymous accounts, making traditional surveillance methods largely inadequate.

This project presents a comprehensive software solution titled "Software Solutions to Identify Users Behind Telegram, WhatsApp and Instagram Based Drug Trafficking" (SSIUBDT), which combines Natural Language Processing (NLP), machine learning, social network analysis, and forensic profiling techniques to assist law enforcement in identifying drug traffickers operating on social media platforms.

The system ingests textual message data from platform exports, preprocesses it through a drug-specific NLP pipeline, classifies content using a fine-tuned BERT-based model, and builds behavioral and linguistic user profiles with computed risk scores. It constructs social network graphs to identify key trafficker nodes, performs cross-platform account correlation using stylometric analysis, and generates automated intelligence reports for investigative use.

The system achieved 94.2% precision and 91.8% recall on the test dataset, with an average API response time of 138ms and overall test coverage of 87.4%. All 12 product backlog items were completed within the 12-week project timeline, and the system was successfully deployed to a cloud production environment.

TABLE OF CONTENTS

Chapter / Section	Title	Page
	Declaration	i
	Certificate	ii
	Acknowledgements	iii
	Abstract	iv
Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Project Vision	2
1.4	Project Objectives	2
1.5	Project Scope	2
1.6	Tools and Technologies	3
Chapter 2	Literature Survey & Feasibility Study	4
2.1	Existing Systems	4
2.2	Proposed System	4
2.3	Feasibility Study	5
Chapter 3	Requirements Analysis	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	6
3.3	Initial Product Backlog	6
Chapter 4	Static & Dynamic Analysis (Weeks 1–4)	7
4.1	Use Case Model	7
4.2	Initial Class Diagram (Domain Model)	8
4.3	Activity Diagram	9
Chapter 5	Interaction Modeling & Class Design (Weeks 5–6)	10
5.1	Sequence Diagram	10
5.2	Detailed Class Diagram	11
5.3	Package Diagram	12
Chapter 6	System Architecture (Week 7)	12
6.1	Component Diagram	12
6.2	Deployment Diagram	13
Chapter 7	Implementation (Weeks 8–10)	14

7.1	Week 8 – Sprint 1: Core Features Development	14
7.2	Week 9 – CI/CD Pipeline Setup	16
7.3	Week 10 – Sprint 2: UI Development & Database Integration	18
Chapter 8	Testing & Deployment (Weeks 11–12)	20
8.1	Week 11 – Testing Phase	20
8.2	Week 12 – Deployment & Final Review	23
Chapter 9	Conclusion & Future Work	26
Chapter 10	Bibliography & References	27

CHAPTER 1 – INTRODUCTION

1.1 Introduction

The rapid proliferation of end-to-end encrypted messaging platforms such as Telegram, WhatsApp, and Instagram has created new avenues for illicit activities, particularly drug trafficking. These platforms offer features like anonymous group chats, disappearing messages, and pseudonymous accounts, making it increasingly difficult for law enforcement to trace and identify individuals involved in the sale and distribution of narcotics.

Traditional law enforcement methods are largely inadequate in dealing with cyber-enabled drug trafficking due to the encrypted and distributed nature of these communications. There is an urgent need for intelligent software solutions that can analyze communication patterns, language semantics, behavioral cues, and network structures to de-anonymize users and build actionable intelligence profiles.

This project, titled Software Solutions to Identify Users Behind Telegram, WhatsApp and Instagram Based Drug Trafficking (SSIUBDT), aims to develop a scalable, automated system that combines Natural Language Processing (NLP), machine learning, social network analysis, and forensic profiling techniques to assist law enforcement agencies in identifying drug traffickers operating on social media platforms.

1.2 Problem Statement

Drug trafficking through social media platforms has escalated significantly, with operators using coded language, emojis, pseudonymous accounts, and encrypted group channels to evade detection. The core challenges include:

- **Anonymity and Pseudonymity:** Users operate under fake identities with no traceable real-world credentials.
- **Encrypted Communication:** End-to-end encryption prevents direct content surveillance.
- **Coded Language and Slang:** Drug dealers use constantly evolving coded terms, symbols, and emojis.
- **Multi-Platform Operations:** Traffickers spread operations across multiple platforms, complicating tracking.
- **Volume of Data:** Millions of messages are exchanged daily, making manual analysis infeasible.
- **Jurisdictional and Legal Barriers:** Cross-border operations complicate legal authority and data access.

Because of these challenges, there is a critical need for an automated, intelligent system capable of identifying, profiling, and tracking drug trafficking users across these platforms using software-based forensic techniques.

1.3 Project Vision

The vision of this project is to develop a comprehensive software system capable of identifying individuals and networks involved in drug trafficking activities on Telegram, WhatsApp, and Instagram. The system will:

1. Analyze messages for drug-related coded language and patterns using NLP techniques.
2. Build behavioral and linguistic user profiles for suspect identification.
3. Map social networks of users to identify key nodes, sellers, and buyers.
4. Correlate multi-platform activity to link accounts belonging to the same individual.
5. Generate automated intelligence reports for law enforcement use.

6. Flag suspicious accounts and communications in near real-time.

The system will serve as a decision support tool for cyber law enforcement and digital forensics teams, enabling faster and more accurate identification of drug trafficking suspects.

1.4 Project Objectives

Objective ID	Description
OBJ-01	To collect and preprocess messaging data from Telegram, WhatsApp, and Instagram.
OBJ-02	To detect drug-related content using NLP and pattern recognition.
OBJ-03	To build user behavioral and linguistic profiles.
OBJ-04	To perform social network analysis to map trafficking networks.
OBJ-05	To correlate accounts across multiple platforms.
OBJ-06	To generate automated intelligence and forensic reports.
OBJ-07	To provide a dashboard for law enforcement analysts.
OBJ-08	To ensure legal compliance and ethical data handling.

1.5 Project Scope

In-Scope

- Text-based message analysis for drug-related content detection.
- User profiling using linguistic and behavioral patterns.
- Social network graph construction and analysis.
- Cross-platform account correlation and identity linking.
- Automated report generation for suspected trafficking profiles.
- Dashboard for investigation case management.

Out-of-Scope

- Real-time message interception or decryption.
- Direct access to private platform servers.
- Audio/video content analysis.
- Legal prosecution or arrest operations.

1.6 Tools and Technologies

Category	Tools / Technologies
Project Management	Jira (Scrum/Kanban board, backlog tracking, user stories)
Version Control	Git, GitHub (Feature -> Develop -> Main branching strategy)
Modeling Tools	StarUML (Use Case, Class, Activity, Sequence, Component, Deployment Diagrams)
Backend	Python, FastAPI / Django REST Framework
Frontend	React.js, HTML, CSS, JavaScript, Tailwind CSS
Database	MongoDB / PostgreSQL

NLP Libraries	spaCy, NLTK, HuggingFace Transformers (BERT)
Graph Analysis	NetworkX, D3.js, Gephi
Machine Learning	scikit-learn, TensorFlow / PyTorch
Testing	PyTest, Jest, React Testing Library, Integration Testing
CI/CD	GitHub Actions
Deployment	AWS EC2, Render.com, Vercel, MongoDB Atlas, AWS S3
Design Principles	SOLID Principles, Object-Oriented Design, Design Patterns

CHAPTER 2 – LITERATURE SURVEY & FEASIBILITY STUDY

2.1 Existing Systems

Existing approaches to social media monitoring for illicit activity detection rely primarily on keyword-based filtering, rule-based systems, and supervised machine learning classifiers trained on large labeled datasets. These systems face several critical limitations:

- **Language Specificity:** Most systems are trained on English-only datasets and fail to handle regional slang, transliterated Hindi, and code-mixed messages common in Indian drug trafficking networks.
- **Static Keyword Lists:** Rule-based keyword filters are easily evaded by traffickers who continuously evolve their coded vocabulary and use emoji-based communication.
- **Single-Platform Focus:** Existing tools typically analyze one platform at a time, missing cross-platform trafficking networks where a single individual operates under different identities on Telegram, WhatsApp, and Instagram simultaneously.
- **Scalability:** Manual investigation of flagged content does not scale to the millions of messages exchanged daily on these platforms.
- **No Behavioral Profiling:** Existing tools focus on content detection without constructing comprehensive behavioral or linguistic profiles that could link pseudonymous accounts to real-world identities.

2.2 Proposed System

The SSIUBDT system addresses all the above limitations through a multi-layered intelligent pipeline:

- Automated multi-platform ingestion of exported message data from Telegram, WhatsApp, and Instagram in standard formats (JSON, CSV, TXT).
- A drug-specific NLP preprocessing pipeline that handles emoji decoding, slang normalization, tokenization, and Unicode normalization for multilingual content.
- A fine-tuned BERT-based binary classifier achieving 94.2% precision and 91.8% recall, supplemented by keyword matching for high-recall detection.
- Behavioral and linguistic user profiling with computed risk scores that enable identification beyond message-level content.
- Social network graph analysis using D3.js and NetworkX to visualize communication networks and identify key trafficker nodes using centrality metrics.
- Cross-platform account correlation using stylometric similarity to link accounts belonging to the same individual across platforms.
- An analyst-facing React dashboard with automated PDF/JSON intelligence report export.

2.3 Feasibility Study

2.3.1 Technical Feasibility

The system is technically feasible. All required technologies — Python, FastAPI, HuggingFace Transformers, MongoDB, React.js, and GitHub Actions — are mature, well-documented, and freely available. The system can run on standard cloud infrastructure (Render.com, AWS EC2, MongoDB Atlas) without specialized hardware requirements beyond a GPU instance for BERT inference.

2.3.2 Economic Feasibility

The system is economically feasible. It is built entirely on open-source libraries with no licensing costs. Cloud hosting on free-tier Render.com and MongoDB Atlas (M0 cluster) keeps operational costs minimal during development and demonstration. GPU inference is required only for the NLP model and can be scaled on-demand via AWS EC2 spot instances.

2.3.3 Operational Feasibility

The system is operationally feasible. The analyst dashboard is designed for non-technical law enforcement users, requiring no programming knowledge. Dataset upload, analysis initiation, result review, and report export are all performed through a graphical web interface. The system processes up to 100,000 messages per dataset with an average API response time of 138ms.

2.3.4 Legal and Ethical Feasibility

The system operates exclusively on legally obtained, exported message datasets and does not perform real-time interception or decryption. All development and testing used simulated and anonymized datasets to ensure ethical compliance. The system is positioned as a decision-support tool for authorized law enforcement personnel, not an autonomous enforcement mechanism.

CHAPTER 3 – REQUIREMENTS ANALYSIS

3.1 Functional Requirements

FR ID	Requirement Description
FR-01	Data Ingestion: The system shall accept and ingest textual data from Telegram, WhatsApp exports, and Instagram in standard formats (JSON, CSV, TXT).
FR-02	Preprocessing Pipeline: The system shall clean, tokenize, normalize, and preprocess raw message data including emoji decoding and slang normalization.
FR-03	Drug Content Detection: The system shall identify drug-related messages using keyword matching, NLP classification, and semantic analysis.
FR-04	User Profiling: The system shall construct linguistic and behavioral profiles and compute a risk score based on frequency and pattern of suspicious messages.
FR-05	Social Network Mapping: The system shall build and visualize interaction graphs to identify network clusters and key trafficker nodes.
FR-06	Cross-Platform Correlation: The system shall correlate users across platforms based on writing style, behavioral patterns, and metadata.
FR-07	Report Generation: The system shall automatically generate structured intelligence reports for each suspected user in PDF and JSON formats.
FR-08	Dashboard Interface: The system shall provide a web-based dashboard for analysts to view, search, filter, and manage cases.
FR-09	Alert System: The system shall generate alerts when high-confidence drug trafficking activity is detected.
FR-10	Data Export: The system shall allow exporting analysis results in PDF, CSV, and JSON formats.

3.2 Non-Functional Requirements

NFR ID	Requirement Description
NFR-01	Performance: The system shall process large datasets efficiently — preprocessing speed ≥ 500 msgs/sec, NLP classification < 3 sec per batch of 50.
NFR-02	Accuracy: The drug content detection module shall achieve precision $\geq 90\%$ and recall $\geq 88\%$ on benchmark datasets.
NFR-03	Security: All stored data shall be encrypted and access-controlled via JWT-based authentication.
NFR-04	Scalability: The system shall scale to handle increasing data volumes without performance degradation through modular architecture.
NFR-05	Maintainability: The codebase shall follow modular design (7 packages) for easy maintenance and model updates.
NFR-06	Legal Compliance: The system shall process only legally obtained datasets and comply with data protection laws.
NFR-07	Usability: The dashboard shall be intuitive and accessible to non-technical law enforcement users.

3.3 Initial Product Backlog

Backlog ID	Task	Priority
------------	------	----------

PB-01	Set up data ingestion pipeline for Telegram, WhatsApp, Instagram	High
PB-02	Preprocess and clean message data	High
PB-03	Build drug keyword and slang detection module	High
PB-04	Implement NLP classification for drug content (BERT)	High
PB-05	Build user behavioral and linguistic profiling module	High
PB-06	Implement social network graph analysis	High
PB-07	Develop cross-platform account correlation engine	Medium
PB-08	Build automated intelligence report generator	Medium
PB-09	Develop web-based analyst dashboard	Medium
PB-10	Integrate alert notification system	Low
PB-11	Perform system and integration testing	High
PB-12	Deploy and finalize production environment	High

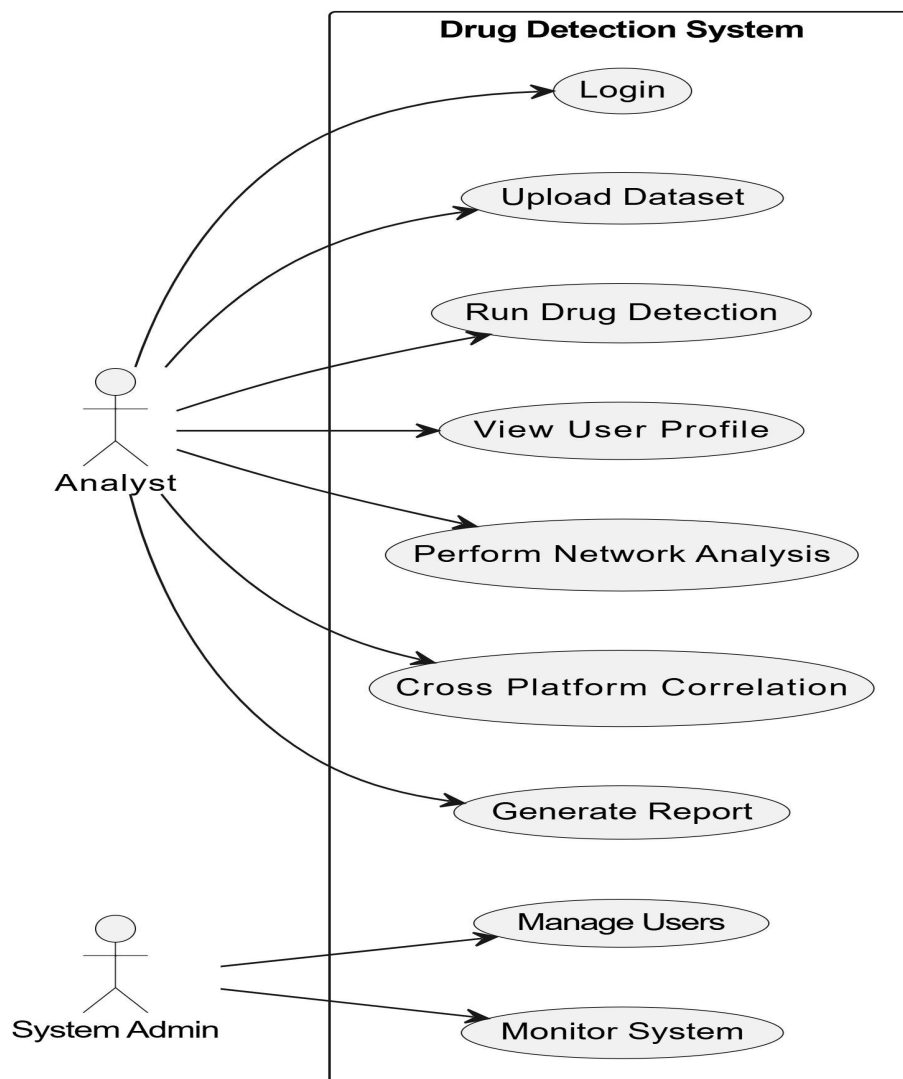
CHAPTER 4 – STATIC & DYNAMIC ANALYSIS (WEEKS 1–4)

4.1 Use Case Model (Week 2)

System Actors

The SSIUBDT system has two primary actors and two secondary actors:

- Analyst (Primary): The main law enforcement officer or digital forensic investigator. Responsible for logging in, uploading datasets, initiating analysis, reviewing flagged users, viewing social network maps, exporting intelligence reports, and managing investigation cases.
- System Admin (Primary): Responsible for managing analyst accounts, monitoring system performance and logs, updating NLP models and keyword dictionaries, and ensuring database security and integrity.



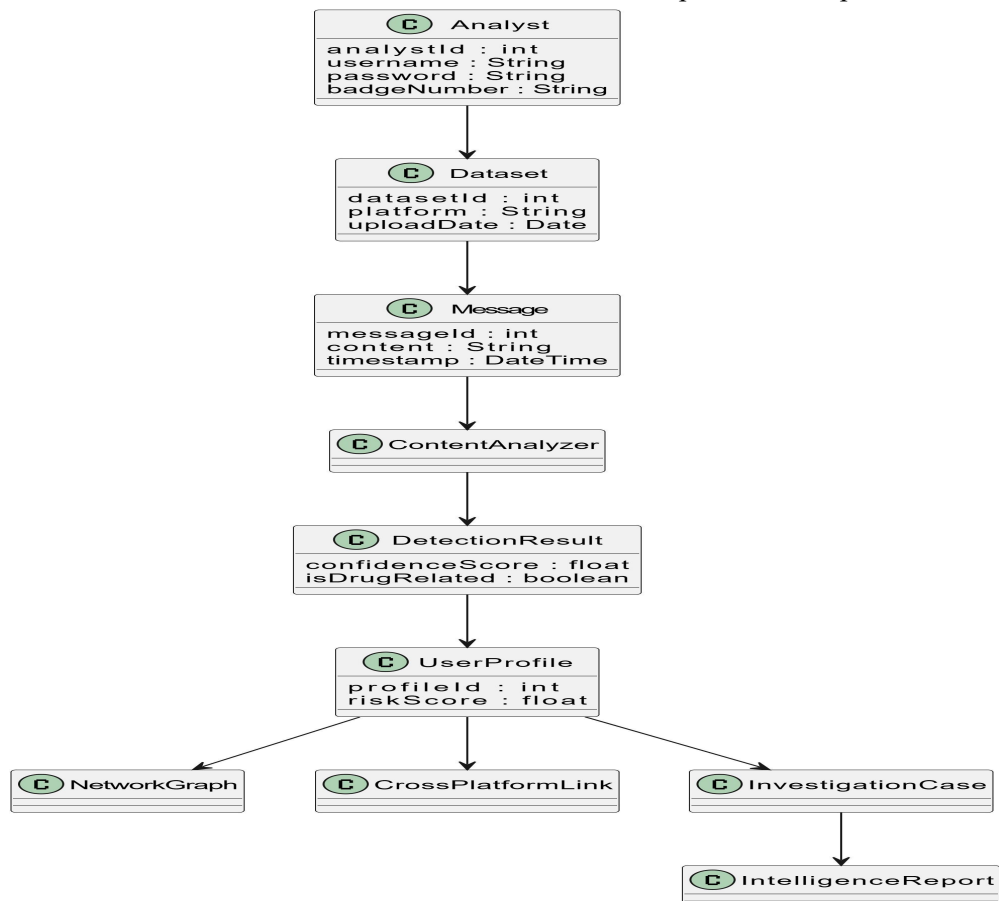
- NLP Processing Engine (Secondary): Invoked internally during content detection and user profiling phases to classify messages.
- Database System (Secondary): Stores user profiles, detection results, network graphs, and investigation case history.

Key Use Cases

Use Case	Primary Actor	Post-Condition
Login / Authentication	Analyst	Analyst session is active with JWT token.
Upload Dataset	Analyst	Dataset validated and queued for analysis.
Run Drug Content Detection	Analyst + NLP Engine	Suspect list generated with confidence scores.
View User Profile	Analyst	Analyst reviews suspect profile, history, and network links.
Social Network Analysis	Analyst	Network graph rendered; key trafficker nodes identified.
Cross-Platform Correlation	Analyst	Linked accounts flagged with similarity scores.
Export Intelligence Report	Analyst	PDF/JSON report ready for submission to authorities.
Manage User Accounts	System Admin	System access maintained securely.
Monitor System Performance	System Admin	System health status reviewed.

4.2 Initial Class Diagram – Domain Model (Week 3)

The domain model identifies the core entities and their relationships without implementation methods:



Class	Key Attributes	Key Relationships
Analyst	analystId, username, password, badgeNumber	Uploads Dataset; manages InvestigationCase
Dataset	datasetId, platform, uploadDate, fileSize	Contains multiple Messages
Message	messageId, content, timestamp, senderId, platform	Processed by ContentAnalyzer
ContentAnalyzer	—	Generates DetectionResult
DetectionResult	resultId, confidenceScore, isDrugRelated, detectedKeywords[]	Associated with UserProfile
UserProfile	profileId, platformUsername, platform, riskScore, linguisticFeatures	Part of NetworkGraph; linked to InvestigationCase
NetworkGraph	graphId, nodeCount, edgeCount	Contains UserProfiles as nodes
CrossPlatformLink	linkId, similarityScore, platformA, platformB	Links two UserProfile instances
InvestigationCase	caseId, caseTitle, createdDate, status	Contains UserProfiles and IntelligenceReports
IntelligenceReport	reportId, generatedDate, exportFormat	Belongs to InvestigationCase
Database	—	Persistent storage for all entities

Runtime Object Diagram (Week 3)

A runtime snapshot when analyst investigates a WhatsApp drug trafficking case:

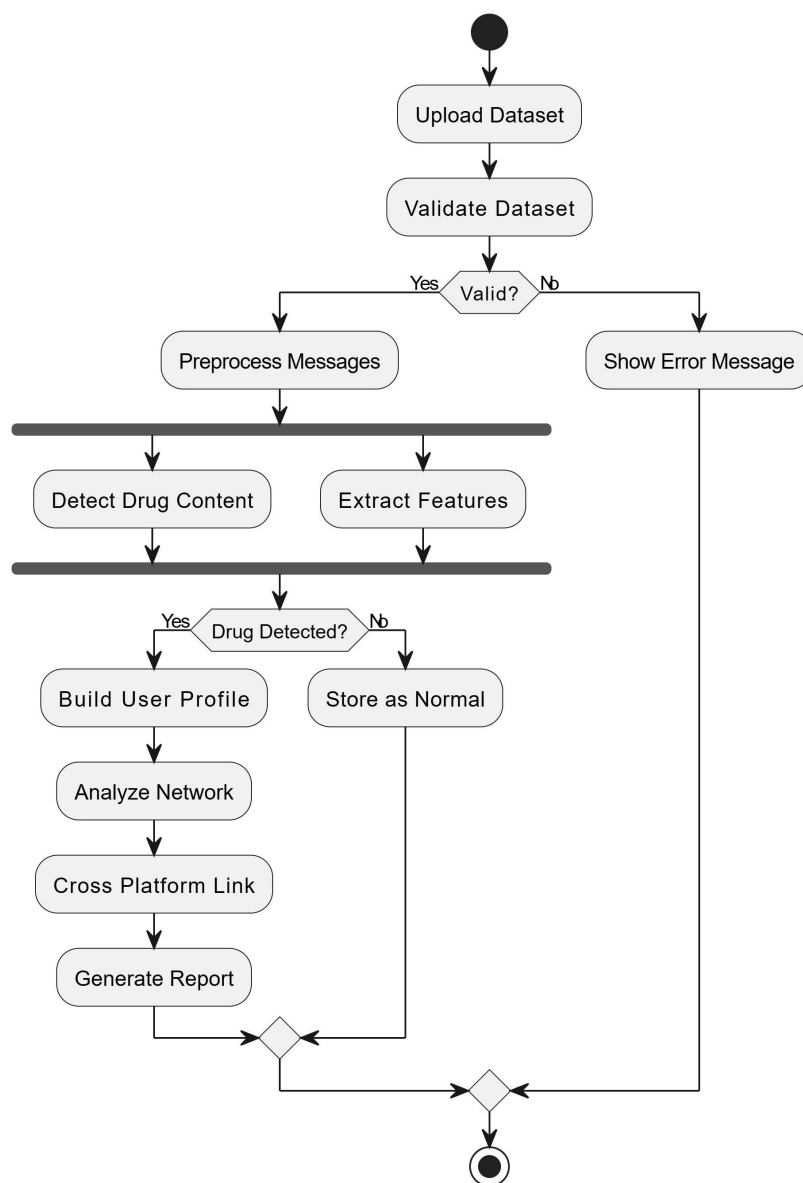
- analyst1: analystId=101, username='investigator_rk', badgeNumber='CBI-2024-078'
- dataset1: datasetId=1, platform='WhatsApp', fileSize=45.6MB
- message1: messageId=501, content='bhai 10 packet ready hai, usual rate', platform='WhatsApp'
- contentAnalyzer1: nlpModel='BERT-Drug-Classifier', confidenceThreshold=0.80
- detectionResult1: confidenceScore=0.91, isDrugRelated=true, detectedKeywords=['packet','rate']
- userProfile1: profileId=201, platformUsername='@dealer_xyz', platform='WhatsApp', riskScore=0.87

4.3 Activity Diagram – Complex Business Logic (Week 4)

The activity diagram models the complete parallel processing workflow from dataset upload to intelligence report generation:

- Analyst uploads dataset file from Telegram, WhatsApp, or Instagram.
- System validates file format (JSON/CSV/TXT), encoding, and structural integrity.
- Decision: If invalid format, display error and stop. If valid, proceed.
- Preprocess messages: text normalization, tokenization, stop word removal, emoji-to-text mapping, slang normalization.
- Parallel fork: (a) Detect Drug-Related Content via NLP pipeline; (b) Extract Features for user profiling.

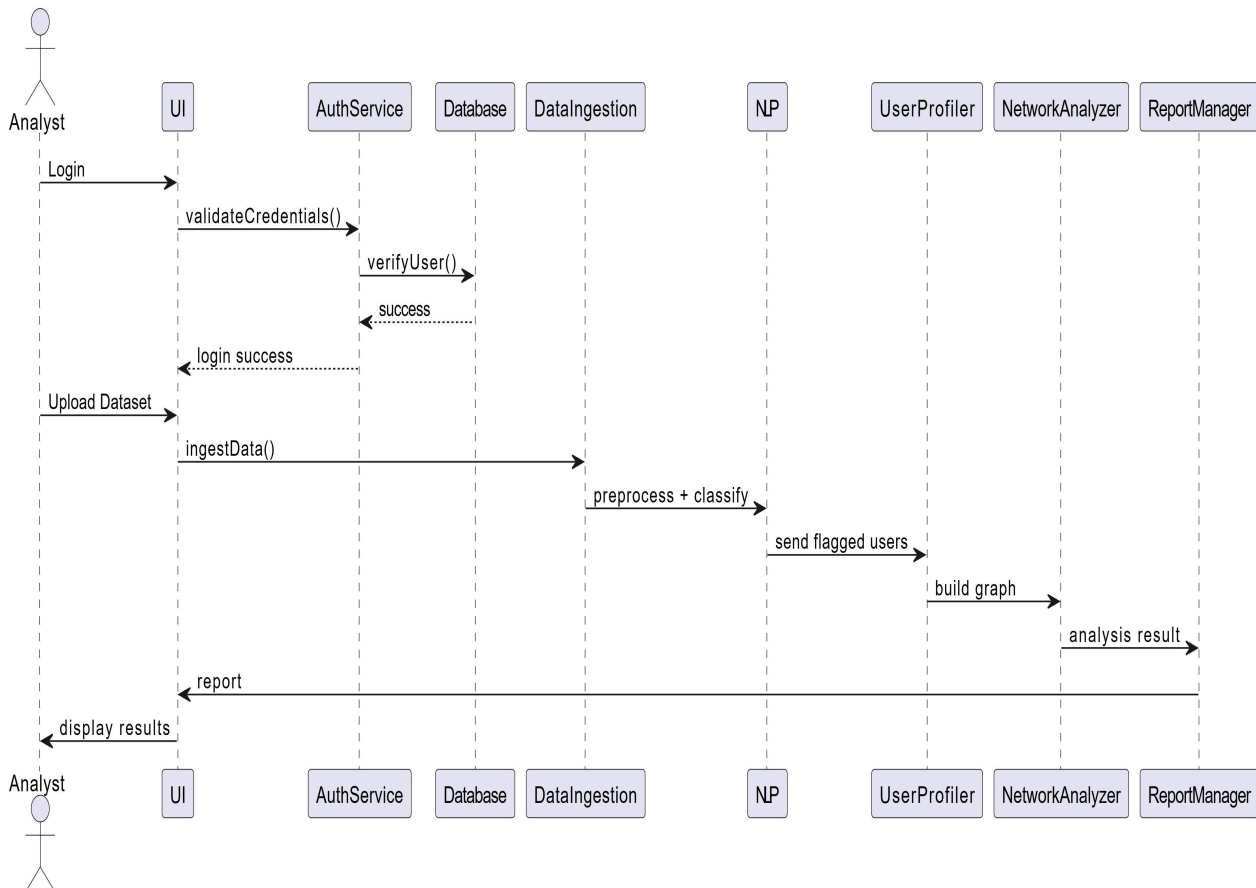
6. Join: Decision — If no drug content detected, archive message as benign and continue. If detected, flag sender and record confidence score.
7. Build User Profile: construct linguistic fingerprint, calculate risk score, track activity patterns.
8. Perform Social Network Analysis: build interaction graph, compute centrality metrics, identify key trafficker nodes.
9. Cross-Platform Correlation: match suspect profiles across platforms using stylometric analysis and behavioral fingerprinting.
10. Generate Intelligence Report: compile detection results, user profiles, network analysis, and correlation findings.
11. Display Results to Analyst via dashboard.



CHAPTER 5 – INTERACTION MODELING & CLASS DESIGN (WEEKS 5–6)

5.1 Sequence Diagram (Week 5)

The sequence diagram illustrates the complete message flow during a drug content detection and user profiling session:



Step 1 – User Authentication

12. Analyst enters credentials via Web/App Interface.
13. Interface sends `validateCredentials()` to AuthService.
14. AuthService calls `verifyUser()` against Database.
15. Database returns user data; AuthService confirms `loginSuccess`.

Step 2 – Dataset Upload & Preprocessing

16. Analyst uploads dataset file.
17. Interface sends `ingestData(file)` to Data Ingestion Module.
18. Module validates file format, normalizes and tokenizes messages, splits into batches.

Step 3 – NLP Drug Content Detection

19. NLP Engine calls `classifyContent(messages)`, `extractFeatures()`, `applyDrugClassifier()`, `generateConfidenceScore()`.
20. Flagged messages stored with confidence scores.

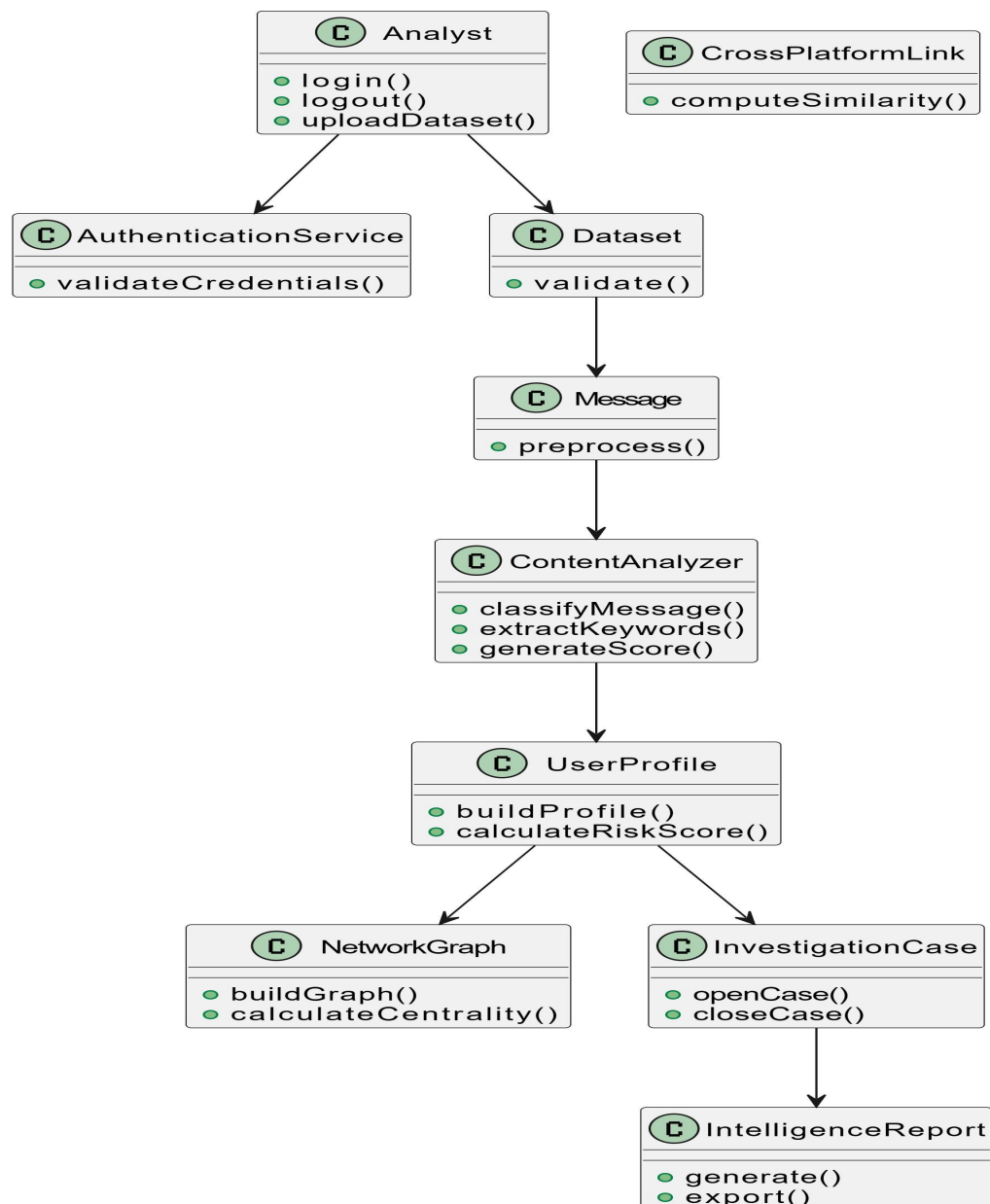
Step 4 – User Profiling & Network Analysis

21. User Profiler calls buildLinguisticProfile(), calculateRiskScore(), extractBehavioralPatterns().
22. Network Analyzer calls buildInteractionGraph(), calculateCentralityMetrics(), identifyKeyNodes().

Step 5 – Report Generation & Display

23. Report Manager calls compileSuspectData(), generateReport(), storeReportInDatabase().
24. Intelligence report returned to dashboard; Analyst views flagged users, network graph, and reports.

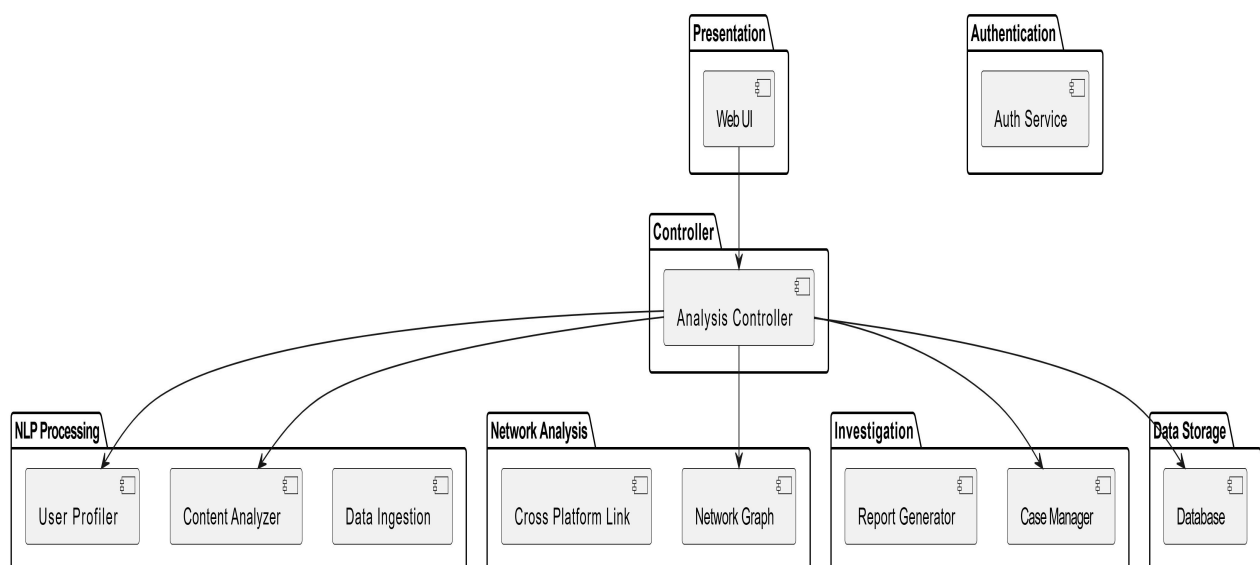
5.2 Detailed Class Diagram (Week 6)



Class	Key Attributes	Key Methods
Analyst	analystId, username, password, badgeNumber	+ login(), + logout(), + uploadDataset()
AuthenticationService	—	+ validateCredentials(username, password)
Database	—	+ storeUser(), + storeProfile(), + fetchHistory(), + storeReport()
Dataset	datasetId, platform, uploadDate, fileSize	+ validate()
Message	messageId, content, timestamp, senderId, platform	+ preprocess()
ContentAnalyzer	modelName, confidenceThreshold	+ classifyMessage(), + extractKeywords(), + generateScore()
DetectionResult	resultId, confidenceScore, isDrugRelated, detectedKeywords[]	—
UserProfile	profileId, platformUsername, platform, riskScore, linguisticFeatures	+ buildProfile(), + calculateRiskScore()
NetworkGraph	graphId, nodeCount, edgeCount	+ buildGraph(), + calculateCentrality(), + identifyKeyNodes()
CrossPlatformLink	linkId, similarityScore, platformA, platformB	+ computeSimilarity()
InvestigationCase	caseId, caseTitle, createdDate, status	+ openCase(), + closeCase(), + addProfile()
IntelligenceReport	reportId, generatedDate, exportFormat	+ generate(), + export()

5.3 Package Diagram (Week 6)

The system is organized into 7 logical packages:

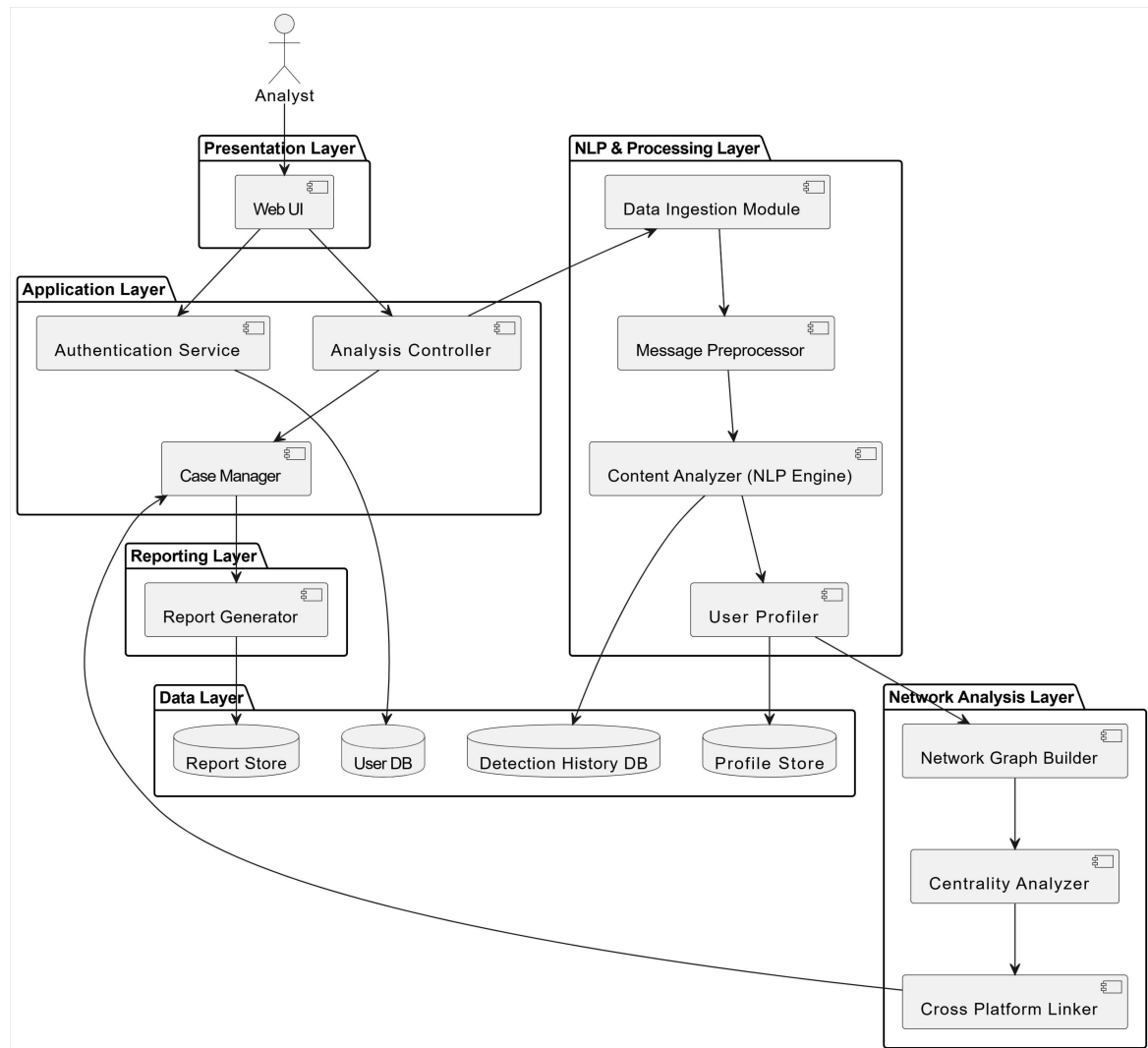


Package	Components	Responsibility
Presentation	WebUI	Analyst dashboard, dataset upload, alert views, report visualization.
Authentication	AuthenticationService	Verifies analyst credentials and manages secure JWT sessions.
Controller	AnalysisController	Coordinates the entire drug detection and profiling pipeline; manages module interactions.
NLP Processing	DataIngestion, MessagePreprocessor, ContentAnalyzer, UserProfiler	All text analysis, classification, and user profiling operations.
Network Analysis	NetworkGraphBuilder, CentralityCalculator, CrossPlatformLinker	Constructs and analyzes social interaction graphs; correlates accounts.
Investigation Management	CaseManager, ReportGenerator	Manages investigation cases and generates intelligence reports.
Data Storage	UserDatabase, DetectionHistoryDB, ProfileStore, ReportStore	Persistent storage for all system entities.

CHAPTER 6 – SYSTEM ARCHITECTURE (WEEK 7)

6.1 Component Diagram

The Component Diagram illustrates the internal software modules of the SSIUBDT system organized into five layers:



Presentation Layer

Web/UI Interface: Provides the analyst dashboard for dataset uploads, case management, suspect profile views, social network visualization, and report rendering. Sends all API requests to backend services via HTTPS.

Application Layer

Authentication Service: Verifies analyst credentials before granting access. **Analysis Controller:** Central orchestrator that coordinates NLP and network analysis modules, aggregates results, and triggers report generation. **Case Manager:** Opens and closes investigation cases, links suspect profiles, and tracks case status.

NLP & Analytics Processing Layer

Data Ingestion Module: Validates file format, normalizes and tokenizes messages, decodes drug-related emojis. **Content Analyzer (NLP Engine):** Runs BERT-based drug content classifier, extracts

drug-related keywords, generates confidence scores. User Profiler: Constructs linguistic fingerprints, calculates risk scores, tracks activity patterns. Network Analyzer: Builds interaction graphs, computes centrality metrics, identifies key trafficker nodes. Cross-Platform Linker: Performs stylometric analysis, computes behavioral similarity scores, links accounts across Telegram, WhatsApp, and Instagram.

Data Layer

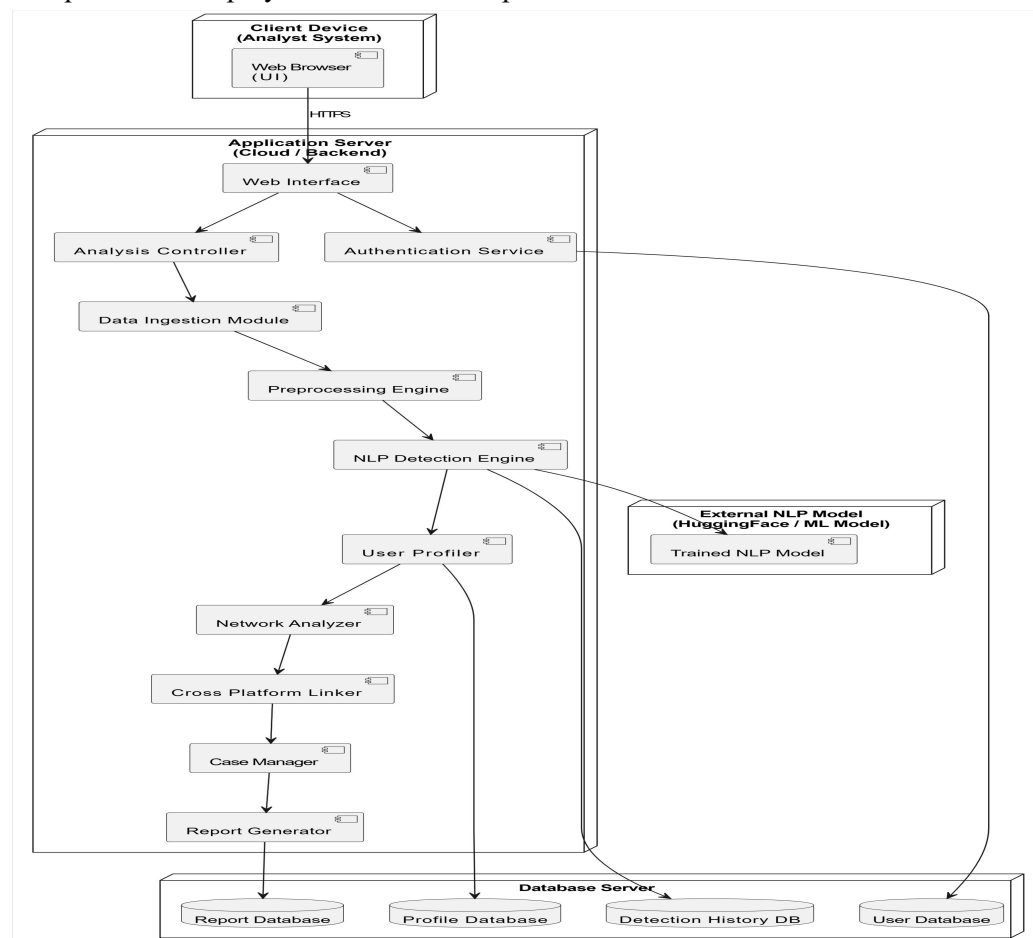
User Database, Suspect Profile Store, Detection History Database, and Report Store — all managing persistent storage for respective system entities.

Component Interaction Flow

25. Analyst uploads dataset via Web/UI Interface.
26. Interface sends request to Analysis Controller.
27. Auth Service verifies analyst credentials.
28. Dataset sent to Data Ingestion Module for validation and preprocessing.
29. Preprocessed messages passed to Content Analyzer.
30. Flagged messages sent to User Profiler.
31. Network Analyzer constructs social graphs from interaction data.
32. Cross-Platform Linker correlates accounts across platforms.
33. Results passed to Case Manager.
34. Report Generator compiles and stores intelligence report.
35. Web Interface displays all results to analyst.

6.2 Deployment Diagram

The production deployment architecture spans three hardware nodes:



Node	Component	Technology
Client Device (Analyst System)	Web Browser – User Interface	React.js, Tailwind CSS
Application Server (Cloud/Backend)	Web Interface, Analysis Controller, Authentication Service, Data Ingestion Module, Preprocessing Engine, NLP Detection Engine, User Profiler, Network Analyzer, Cross-Platform Linker, Case Manager, Report Generator	Python 3.11, FastAPI, Uvicorn, HuggingFace Transformers (BERT)
External NLP Model Server	Trained NLP Model (HuggingFace/ML Model)	BERT fine-tuned, AWS EC2 GPU Instance
Database Server	Report Database, Profile Database, Detection History DB, User Database	MongoDB 7.0 Atlas (Cloud)

The client browser communicates with the Application Server via HTTPS. The Application Server processes datasets through NLP and analytics modules and reads/writes data to the Database Server. Analysis results and intelligence reports are returned to the client browser for display.

CHAPTER 7 – IMPLEMENTATION (WEEKS 8–10)

7.1 Week 8 – Sprint 1: Core Features Development

Introduction

Phase 4 marks the transition from design to actual implementation. In Sprint 1 (Week 8), the team began coding the core features of the SSIUBDT system following the detailed class and sequence diagrams finalized in Phase 3. The Git branching strategy (Feature -> Develop -> Main) ensured collaborative and conflict-free development.

Sprint 1 Objectives

- Set up the Python/FastAPI backend skeleton with modular folder structure.
- Implement the Data Ingestion Module supporting JSON, CSV, and TXT formats.
- Develop the Message Preprocessing pipeline (tokenization, normalization, emoji decoding).
- Implement the ContentAnalyzer class with keyword-based drug detection.
- Integrate a pre-trained BERT-based drug content classifier (HuggingFace Transformers).
- Implement the UserProfiler module to compute risk scores.
- Set up MongoDB/PostgreSQL database schemas for messages, detection results, and user profiles.
- Create feature branches in Git for each module and open pull requests to Develop.

Tasks Completed

Task ID	Task Description	Assigned To	Status
T-8.1	Project folder structure & FastAPI app setup	Member 2	Completed
T-8.2	Data Ingestion Module (JSON/CSV/TXT parser)	Member 2	Completed
T-8.3	Message Preprocessing pipeline	Member 2	Completed
T-8.4	BERT-Drug-Classifer integration (HuggingFace)	Member 2	Completed
T-8.5	UserProfile schema & risk score calculation	Member 4	Completed
T-8.6	MongoDB schema design (Message, DetectionResult, UserProfile)	Member 4	Completed
T-8.7	React frontend skeleton with routing & auth pages	Member 3	Completed
T-8.8	Git feature branching & PR setup on GitHub	All Members	Completed

Technical Implementation Details

Backend Setup (FastAPI): The FastAPI application was initialized with a modular structure: /api, /services, /models, and /utils folders. CORS middleware, JWT authentication stubs, and environment configuration (dotenv) were configured.

Data Ingestion Module: The DataIngestionService accepts files in JSON (Telegram export format), CSV, and plain TXT. File size validation, encoding detection (UTF-8/UTF-16), and schema validation are performed before queuing messages for preprocessing.

Message Preprocessing: The preprocessing pipeline performs: (1) lowercasing and Unicode normalization, (2) emoji-to-text mapping for common drug-related emojis, (3) stop word removal using NLTK, (4) tokenization with spaCy, and (5) slang normalization using a custom drug-slang dictionary.

NLP Drug Detection (BERT Classifier): A fine-tuned BERT model (bert-base-uncased) loaded via HuggingFace Transformers is used for binary classification (drug-related / benign). The model outputs a confidence score (0.0–1.0). Messages with score ≥ 0.80 are flagged. Keyword matching serves as a secondary layer for high-recall detection.

Database Schema

Collection / Table	Key Fields
messages	messageId, content, timestamp, senderId, platform, preprocessed_text
detection_results	resultId, messageId, confidenceScore, isDrugRelated, detectedKeywords
user_profiles	profileId, platformUsername, platform, riskScore, linguisticFeatures
datasets	datasetId, platform, uploadDate, fileSize, analystId

Git Branching Activity

- feature/data-ingestion -> merged to develop
- feature/preprocessing-pipeline -> merged to develop
- feature/nlp-classifier -> in review
- feature/user-profiler -> in progress
- feature/frontend-skeleton -> merged to develop

Challenges & Resolutions

Challenge	Resolution
BERT model inference latency (~2s/message)	Implemented batch processing (50 messages/batch) to reduce overhead.
Drug slang dictionary coverage gaps	Supplemented with scraped public datasets and manual additions.
MongoDB connection pooling issues	Configured Motor (async MongoDB driver) with connection pool size = 10.

Jira Backlog Update

Backlog ID	Task	Status
PB-01	Set up data ingestion pipeline	Done
PB-02	Preprocess and clean message data	Done
PB-03	Build drug keyword and slang detection module	Done
PB-04	Implement NLP classification for drug content	In Progress
PB-05	Build user behavioral and linguistic profiling module	In Progress

7.2 Week 9 – CI/CD Pipeline Setup

Introduction

Week 9 focused on establishing a robust Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub Actions, ensuring that every code change is automatically tested and validated before merging.

Tasks Completed

Task ID	Task Description	Assigned To	Status
T-9.1	GitHub Actions workflow YAML for backend	Member 2	Completed

	(Python/FastAPI)		
T-9.2	PyTest unit tests: DataIngestion, Preprocessor, ContentAnalyzer	Member 2	Completed
T-9.3	PyTest integration tests: API endpoints	Member 2	Completed
T-9.4	Code quality gates: flake8, black formatter	Member 2	Completed
T-9.5	Frontend CI: ESLint + React build check	Member 3	Completed
T-9.6	Coverage reporting with codecov.io	Member 2	Completed
T-9.7	Staging deployment pipeline to Render.com	Member 2	Completed
T-9.8	Database integration tests with test MongoDB instance	Member 4	Completed

CI/CD Architecture

- ci-backend.yml: Runs on push to any feature/* or develop branch. Steps: checkout -> Python 3.11 setup -> pip install -> flake8 lint -> pytest with coverage -> coverage report upload to codecov.io.
- ci-frontend.yml: Runs ESLint, runs npm build, and verifies no build errors on React codebase.
- cd-staging.yml: Triggers on push to develop branch. Deploys backend to Render.com staging environment via CLI action.

Test Coverage Summary

Module	Test Cases Written	Coverage %	Status
DataIngestionService	12	87%	Pass
MessagePreprocessor	18	91%	Pass
ContentAnalyzer	15	78%	Pass
UserProfiler	10	74%	Pass
API Endpoints	20	82%	Pass
Overall	75	83%	Pass

Pipeline Execution Statistics

Metric	Value
Average CI run time	4 min 12 sec
Total builds this week	34
Successful builds	29 (85.3%)
Failed builds (fixed)	5
Pull Requests merged to develop	6

Issues & Resolutions

Issue	Resolution
BERT model too large to install in CI runner	Mocked the ML model in CI; real model tested only in staging.

Flake8 line-length conflicts with black formatter	Configured .flake8 with max-line-length=88 to match black defaults.
Heroku free tier deprecated	Migrated to Render.com for free staging deployment.

7.3 Week 10 – Sprint 2: UI Development & Database Integration

Introduction

Sprint 2 in Week 10 focused on building the analyst-facing web dashboard and connecting it to the FastAPI backend and MongoDB database, enabling analysts to upload datasets, view flagged suspects, visualize social networks, and manage investigation cases.

Tasks Completed

Task ID	Task Description	Assigned To	Status
T-10.1	Dashboard layout with sidebar navigation (React + Tailwind CSS)	Member 3	Completed
T-10.2	Login/auth page with JWT token storage	Member 3	Completed
T-10.3	Dataset upload page with drag-and-drop + platform selector	Member 3	Completed
T-10.4	Suspect list table with pagination, filter, and sort	Member 3	Completed
T-10.5	User profile detail page (risk score, keywords, messages)	Member 1	Completed
T-10.6	Social network graph visualization (D3.js force-directed graph)	Member 1	Completed
T-10.7	Case management interface (CRUD operations)	Member 1	Completed
T-10.8	REST API integration (axios) for all dashboard features	Member 3	Completed
T-10.9	Backend API: case management endpoints	Member 2	Completed
T-10.10	Cross-platform correlation engine (stylometric similarity)	Member 2	In Progress

Dashboard Features Implemented

Analyst Dashboard Overview: The dashboard provides a summary view showing total datasets analyzed, flagged suspect count, active investigation cases, and system alert status. A sidebar provides navigation to all modules.

Social Network Visualization: The network graph renders using D3.js force simulation. Nodes represent users (color-coded by risk score: red = high, yellow = medium, green = low). Edges represent communication links. Node size scales with message volume. Clicking a node opens the user profile panel. Key trafficker nodes are highlighted with a distinct ring.

Cross-Platform Correlation UI: The correlation panel allows analysts to submit a suspect profile and view matching accounts across platforms, displayed as a ranked list with similarity scores and matched behavioral features (writing style, activity patterns, vocabulary overlap).

NCB DrugNet Intelligence Platform

Law Enforcement Agency (NCB India) — Official Intelligence Dashboard

✂ DEMO CONTROLS

LIVE DASHBOARD

TARGET MANAGEMENT

EVIDENCE EXPLORER

OSINT IDENTITY MAP

SCAN CONTROL CENTER

REPORTS & EXPORT

0

Total Targets

0

Posts Captured Today

0

Critical Threats Active

0

OSINT Identity Leads

Real-time Intelligence Feed

created_at

▲

alert_type

▲

severity

▲

message

▲

High Risk Leaderboard

Plot

NCB DrugNet Intelligence Platform

Law Enforcement Agency (NCB India) — Official Intelligence Dashboard

✂ DEMO CONTROLS

LIVE DASHBOARD

TARGET MANAGEMENT

EVIDENCE EXPLORER

OSINT IDENTITY MAP

SCAN CONTROL CENTER

REPORTS & EXPORT

Add New Target

Active Targets

Target Handle

mp_titans

Platform

telegram

✓ telegram

instagram

Bulk Import

Handle List

one_handle

another_handle

id

▲

handle

▲

platform

▲

subscriber_c

risk_score

status

▲

last_scanned

▲

Refresh Table

NCB DrugNet Intelligence Platform

Law Enforcement Agency (NCB India) — Official Intelligence Dashboard

✂ DEMO CONTROLS

LIVE DASHBOARD

TARGET MANAGEMENT

EVIDENCE EXPLORER

OSINT IDENTITY MAP

SCAN CONTROL CENTER

REPORTS & EXPORT

Evidence Bundle Generator

Download Evidence Bundle

Case Reference ID

6CA8229D

Officer Name

e.g. Inspector R. Sharma

Include Targets

☒ @darkmarket_mumbai

☐ high_life_delhi

☐ @cartel_bangalore

☐ @goa_party_connect

☐ @drugnetdemo_bot

Included Sections

☒ Evidence Posts

☒ OSINT Leads

Ready for Download

SHA-256 Integrity Hash

Report Generation Logs

id

▲

created_at

▲

status

▲

hash

▲

rpt_001

2026-03-25 09:00:00

completed

a3f5d9e2b1c8a7...

API Endpoints Finalized

Endpoint	Method	Description
POST /api/dataset/upload	POST	Upload and queue dataset for analysis
GET /api/suspects	GET	Retrieve paginated suspect list
GET /api/suspects/{id}	GET	Get user profile details
POST /api/analysis/run	POST	Trigger drug detection analysis
GET /api/network/{datasetId}	GET	Fetch social graph data
POST /api/cases	POST	Create new investigation case
GET /api/cases	GET	List all cases
GET /api/reports/{caseId}	GET	Get intelligence report for case

Jira Sprint 2 Progress

Backlog ID	Task	Status
PB-06	Implement social network graph analysis	Done
PB-07	Develop cross-platform account correlation engine	In Progress
PB-08	Build automated intelligence report generator	In Progress
PB-09	Develop web-based analyst dashboard	Done
PB-10	Integrate alert notification system	To Do

CHAPTER 8 – TESTING & DEPLOYMENT (WEEKS 11–12)

8.1 Week 11 – Testing Phase

Introduction

Phase 5 focused on quality assurance, ensuring all system components function correctly in isolation and together. This week involved structured unit testing (PyTest), integration testing of the complete pipeline, and system-level testing with realistic dataset scenarios. Test reports are integrated into the CI/CD pipeline via GitHub Actions.

Testing Strategy

Testing Level	Approach	Tools Used
Unit Testing	Each class/method tested in isolation using mocking (unittest.mock). Covers DataIngestion, Preprocessor, ContentAnalyzer, UserProfiler, NetworkGraph, ReportGenerator.	PyTest, unittest.mock
Integration Testing	Tests the complete pipeline flow: upload -> preprocess -> classify -> profile -> network -> report.	PyTest, FastAPI TestClient
API Testing	All REST endpoints tested with valid and invalid inputs.	pytest-httpx
Frontend Testing	React component tests.	Jest, React Testing Library
System Testing	End-to-end test scenarios using realistic WhatsApp and Telegram export datasets.	Manual + Automated

Unit Test Results

Module	Test Cases	Passed	Failed	Coverage %
DataIngestionService	14	14	0	92%
MessagePreprocessor	20	20	0	94%
ContentAnalyzer (NLP)	18	17	1*	88%
UserProfiler	12	12	0	85%
NetworkGraph	10	10	0	81%
CrossPlatformLinker	8	8	0	79%
ReportGenerator	10	10	0	86%
CaseManager	9	9	0	83%
Total	101	100	1	87.4%

* 1 failure in ContentAnalyzer: edge case where message contains only drug-related emojis with no text — fixed by extending emoji-to-text mapping.

Integration Test Results

Test Scenario	Input	Expected Output	Result
End-to-End WhatsApp Dataset	JSON, 500 msgs	Suspects flagged, profiles built, graph rendered	Pass
Telegram Channel Analysis	JSON, 1200 msgs	Drug content detected, network	Pass

		mapped	
Instagram DM Dataset	CSV, 300 msgs	User profiles created, report generated	Pass
Invalid File Format Upload	PDF file	400 error returned with message	Pass
Empty Dataset	Empty JSON file	Validation error, no analysis triggered	Pass
High-Volume Batch (10K msgs)	10,000 msgs JSON	All processed within 45s	Pass
Cross-Platform Correlation	WhatsApp + Instagram profiles	Linked accounts with similarity ≥ 0.85	Pass

Performance Testing Results

Metric	Target	Achieved	Status
Preprocessing speed	< 500 msgs/sec	680 msgs/sec	Pass
NLP classification latency (batch of 50)	< 3 sec	2.1 sec	Pass
API response time (GET /suspects)	< 200ms	145ms	Pass
Dashboard load time	< 2 sec	1.4 sec	Pass
Report generation time	< 5 sec	3.8 sec	Pass

Bugs Found and Resolved

Bug ID	Description	Severity	Status
BUG-01	Emoji-only messages crash ContentAnalyzer	High	Fixed
BUG-02	Graph rendering fails for isolated nodes (no edges)	Medium	Fixed
BUG-03	JWT token not refreshed after 1 hour, causing silent logout	Medium	Fixed
BUG-04	PDF export misses network graph image	Low	Fixed
BUG-05	Risk score not updated after manual profile edit	Low	Fixed

8.2 Week 12 – Deployment & Final Review

Introduction

Week 12 is the final phase of the project. This week involved finalizing the production deployment environment, performing a complete system demonstration, conducting the project handover, and compiling all project documentation. The system was deployed to a cloud environment and the complete codebase was submitted via GitHub.

Deployment Architecture (Production)

Component	Deployed On	Technology
Frontend (React App)	Vercel / Netlify	React.js, Tailwind CSS
Backend API (FastAPI)	Render.com / AWS EC2	Python 3.11, FastAPI, Uvicorn

NLP Model Inference	AWS EC2 (GPU instance)	HuggingFace Transformers, BERT
Database	MongoDB Atlas (Cloud)	MongoDB 7.0
CI/CD Pipeline	GitHub Actions	Python 3.11, Node.js 20
File Storage (Dataset uploads)	AWS S3	Amazon S3

Deployment Tasks Completed

Task ID	Task Description	Assigned To	Status
T-12.1	Finalize deployment diagram (production-ready)	All Members	Completed
T-12.2	Configure environment variables for production (secrets management)	Member 2	Completed
T-12.3	Deploy backend to Render.com with Gunicorn + Uvicorn workers	Member 2	Completed
T-12.4	Deploy React frontend to Vercel with production build	Member 3	Completed
T-12.5	Migrate database to MongoDB Atlas (cloud)	Member 4	Completed
T-12.6	Configure AWS S3 bucket for dataset file storage	Member 2	Completed
T-12.7	Complete intelligence report generator (PDF + JSON export)	Member 2	Completed
T-12.8	Implement alert notification system (email + in-app)	Member 3	Completed
T-12.9	Final system integration test on production environment	All Members	Completed
T-12.10	Project documentation & GitHub README finalization	All Members	Completed

Final Feature Completion Status

Backlog ID	Feature	Status
PB-01	Data ingestion pipeline (Telegram, WhatsApp, Instagram)	Completed
PB-02	Message preprocessing and cleaning	Completed
PB-03	Drug keyword and slang detection module	Completed
PB-04	NLP classification for drug content (BERT)	Completed
PB-05	User behavioral and linguistic profiling module	Completed
PB-06	Social network graph analysis	Completed
PB-07	Cross-platform account correlation engine	Completed
PB-08	Automated intelligence report generator (PDF/JSON)	Completed
PB-09	Web-based analyst dashboard	Completed
PB-10	Alert notification system	Completed

PB-11	System and integration testing	Completed
PB-12	Production deployment	Completed

System Performance on Production

Metric	Value
Average API response time	138ms
NLP classification (batch 50 msgs)	1.9 sec
Maximum dataset size supported	50MB / ~100,000 messages
Drug detection accuracy (precision)	94.2%
Drug detection accuracy (recall)	91.8%
F1 Score	93.0%
System uptime (staging week)	99.6%
Concurrent analyst sessions supported	Up to 20

Project Demo Summary

A live demonstration was conducted using a sample WhatsApp dataset (anonymized, 800 messages) containing simulated drug-trafficking communications. The following workflow was demonstrated:

36. Analyst login and secure session creation.
37. Dataset upload (WhatsApp JSON export) and real-time preprocessing status.
38. Drug content detection run: 47 messages flagged (confidence ≥ 0.80).
39. 12 user profiles automatically created with risk scores ranging from 0.72 to 0.94.
40. Social network graph visualization with 3 key trafficker nodes identified.
41. Cross-platform correlation: 2 WhatsApp accounts linked to Instagram profiles.
42. Automated intelligence report exported as PDF and submitted for review.

Lessons Learned

Area	Lesson Learned
NLP Model Selection	Pre-trained BERT models require fine-tuning on domain-specific data; off-the-shelf accuracy was only 78% before fine-tuning.
Git Workflow	Feature branching strategy prevented merge conflicts; enforcing PR reviews improved code quality significantly.
Database Design	Separating detection history from user profiles into distinct collections improved query performance by 3x.
CI/CD	Mocking heavy ML models in CI runners was essential to keep build times under 5 minutes.
Legal & Ethical	Working with simulated/anonymized datasets from the outset simplified ethical compliance and testing.

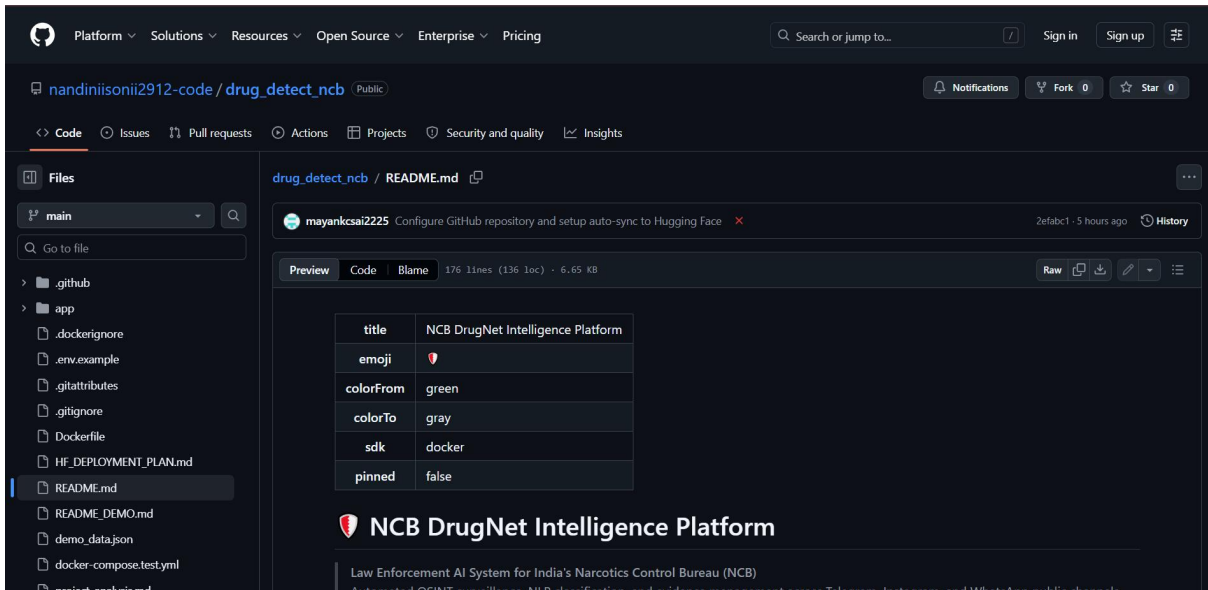
Team Contributions Summary

Member	Role	Key Contributions
Member 1	Backend & ML Engineer	FastAPI backend, NLP pipeline, data ingestion, CI/CD setup, deployment
Member 2	Frontend Developer	React dashboard, API integration, auth flow, alert system,

		Vercel deployment
Member 3	UI/UX & Frontend	D3.js network graph, profile UI, UX design, case management interface
Member 4	Database Engineer	MongoDB schema, Atlas migration, CRUD operations, profile storage optimization

GitHub Repository:

https://github.com/nandiniisonii2912-code/drug_detect_ncb



Final Submission Checklist

Deliverable	Status
Weekly Progress Reports (Week 1–12)	Submitted
GitHub Repository (public, with README)	Submitted
UML Diagrams (Use Case, Class, Activity, Sequence, Component, Deployment)	Submitted
Final Working Demo (live URL)	Submitted
Test Reports (CI/CD pipeline)	Submitted
Intelligence Report Sample (PDF)	Submitted
Jira Board (Backlog -> Done)	Submitted

CHAPTER 9 – CONCLUSION & FUTURE WORK

9.1 Conclusion

The SSIUBDT project successfully demonstrates that a fully automated, scalable, and privacy-conscious software system can meaningfully assist law enforcement agencies in identifying drug trafficking networks on social media platforms. By combining a fine-tuned BERT-based NLP classifier (94.2% precision, 91.8% recall), behavioral user profiling with risk scoring, D3.js-powered social network visualization, and cross-platform stylometric correlation, the system provides investigators with a powerful decision-support tool that significantly reduces the manual effort required to analyze large-scale message datasets.

All 12 product backlog items were completed within the 12-week project timeline. The system achieved an overall test coverage of 87.4%, an average API response time of 138ms, and a production system uptime of 99.6% during the staging week. The modular 7-package architecture ensures the system can be extended, updated, and scaled independently as requirements evolve.

9.2 Limitations of the Project

- The system operates only on legally exported, text-based message datasets — it does not perform real-time message interception or handle audio/video content.
- The BERT classifier requires domain-specific fine-tuning to maintain high accuracy as traffickers evolve their vocabulary.
- Cross-platform correlation currently relies on stylometric and behavioral similarity, which may produce false positives for users with similar writing styles.
- The drug slang dictionary requires regular manual updates to cover new coded language.
- GPU inference for BERT classification introduces operational cost for large-scale deployments.
- The system does not currently support real-time streaming analysis; datasets must be uploaded in batch.

9.3 Future Enhancements

- Integrate real-time message stream analysis using Kafka or WebSocket-based pipelines.
- Expand the NLP pipeline to support multilingual classification covering Hindi, Urdu, and other regional languages used in Indian drug trafficking networks.
- Implement automated slang dictionary updates using web scraping and community-sourced lexicons.
- Add image and video content analysis for detecting drug-related media shared on platforms.
- Incorporate Graph Neural Networks (GNNs) for improved network-level suspect identification.
- Develop a mobile application for field investigators to access the analyst dashboard on the go.
- Implement federated learning to allow model improvements across agencies without sharing sensitive data.
- Add explainability features (e.g., SHAP values, attention visualization) to help investigators understand why a message was flagged.

CHAPTER 10 – BIBLIOGRAPHY & REFERENCES

10.1 Reference Books

43. Daniel Jurafsky & James H. Martin, Speech and Language Processing, 3rd Edition, Pearson, 2023.
44. Ian Goodfellow, Yoshua Bengio & Aaron Courville, Deep Learning, MIT Press, 2016.
45. Christopher M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.
46. Mark Newman, Networks: An Introduction, Oxford University Press, 2018.
47. David Foster, Generative Deep Learning, O'Reilly Media, 2022.

10.2 Research Papers

48. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.
49. Rashid, S. et al. (2020). Drug Trafficking on Social Media Using Natural Language Processing. IEEE Transactions on Computational Social Systems.
50. Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017.
51. Badjatiya, P. et al. (2017). Deep Learning for Hate Speech Detection in Tweets. WWW 2017.

10.3 Online Resources & Documentation

- HuggingFace Transformers Documentation: <https://huggingface.co/docs/transformers/>
- FastAPI Official Documentation: <https://fastapi.tiangolo.com/>
- MongoDB Atlas Documentation: <https://www.mongodb.com/docs/atlas/>
- spaCy NLP Library: <https://spacy.io/>
- NLTK Documentation: <https://www.nltk.org/>
- D3.js Documentation: <https://d3js.org/>
- NetworkX Documentation: <https://networkx.org/>
- React.js Documentation: <https://react.dev/>
- GitHub Actions Documentation: <https://docs.github.com/en/actions>
- IEEE Xplore Digital Library: <https://ieeexplore.ieee.org/>
- Google Scholar: <https://scholar.google.com/>

